EE-170L COMPUTER PROGRAMMING LAB

LAB REPORT

Lab No:	05
Title:	The while Loop in C++
Subject:	Computer Programming

Name:	Kaleem Ullah
Registration No:	BF25NWELE0661
Section:	B

Table of Contents

1. Introduction

This lab report covers the fundamental looping constructs in C++ programming, specifically focusing on **the while loop**, **the nested while loop**, and **the do...while loop**. Loops are essential control flow statements that allow code to be executed repeatedly based on a given condition. Understanding these constructs is crucial for writing efficient and structured programs.

2. Objectives

The objectives of this lab are to learn and apply:

• The while loop and its syntax

• The nested while loop for complex patterns

• The do...while loop and its execution flow

• Practical implementation of all three loop types

3. Theory

3.1 The while Loop

A **while loop** is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop consists of a block of code and a condition. The condition is first evaluated, and if the condition is **true**, the code within the block is then executed. This repeats until the condition becomes **false**.

Syntax:

```
initialization
while (condition) {
    statements;
    update variable value;
}
```

Example: Countdown from 5 to 1

```
#include <iostream>
using namespace std;

int main() {
    int i = 5;          // loop control variable
    while (i > 0) {
        cout << i << "\n";
        i--;            // decrement
    }
    cout << "outside loop now";
    return 0;
}
```

Example: Print numbers until -1 is entered

```
#include <iostream>
using namespace std;

int main() {
    int num = 5;
    while (num != -1) {
        cout << "Enter a number to print: -1 to quit\n";
        cin >> num;
        cout << "You entered: " << num << endl;
    }
    cout << "Outside loop now";
    return 0;
}
```

3.2 The Nested while Loop

A **nested while loop** is a while loop placed inside another while loop. The inner loop completes all its iterations for each single iteration of the outer loop. Nested loops are commonly used for generating patterns and multi-dimensional data processing.

Example: Pattern Printing

```
#include <iostream>
using namespace std;

int main() {
    int num = 5;
    while (num >= 1) {
        int nested = num;
        while (nested >= 1) {
            cout << nested << " ";
            nested--;
        }
        cout << "\n";
        num--;
    }
    return 0;
}
```

3.3 The do...while Loop

The **do...while loop** is a variant of the while loop with one important difference: the body of the do...while loop is executed **once before** the condition is checked. This guarantees that the loop body executes at least one time. After the first execution, the condition is evaluated, and if true, the loop continues; otherwise, it terminates.

Syntax:

```
initialization
do {
    statements;
    update variable value;
} while (condition);
```

Example: Password Validation

```
#include <iostream>
using namespace std;

int main() {
    int password;
    do {
        cout << "Enter password (1234 to exit): ";
        cin >> password;
        if (password != 1234) {
            cout << "Wrong password! Try again.\n";
        }
    } while (password != 1234);
    cout << "Access Granted!" << endl;
    return 0;
}
```

4. Lab Tasks

Task 1: while Loop - Display Numbers from 1 to 10

Problem Statement: Write a C++ program that uses a while loop to display numbers from 1 to 10, with each number on a separate line.

Source Code:

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;              // initialization

    while (i <= 10) {       // condition
        cout << i << endl;  // display number
        i++;                // increment
    }

    return 0;
}
```

Expected Output:

```
1
2
3
4
5
6
7
8
9
10
```

Task 2: do...while Loop - Display Numbers from 1 to 10

Problem Statement: Repeat Task 1 using a do...while loop to display numbers from 1 to 10.

Source Code:

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;              // initialization

    do {
        cout << i << endl;  // display number
        i++;                // increment
    } while (i <= 10);      // condition checked after execution

    return 0;
}
```

Expected Output:

```
1
2
```

```
3
4
5
6
7
8
9
10
```

Task 3: Sum using while Loop

Problem Statement: Write a C++ program that prompts the user to enter a positive integer. Use a while loop to calculate the sum of all numbers from 1 to the entered integer, and display the calculated sum.

Source Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;                  // user input
    int sum = 0;            // accumulator
    int i = 1;              // counter

    cout << "Enter a positive integer: ";
    cin >> n;

    while (i <= n) {
        sum = sum + i;      // add current number to sum
        i++;                // increment counter
    }

    cout << "Sum of numbers from 1 to " << n
         << " is: " << sum << endl;

    return 0;
}
```

Sample Output:

```
Enter a positive integer: 5
Sum of numbers from 1 to 5 is: 15

Enter a positive integer: 10
Sum of numbers from 1 to 10 is: 55
```

5. Conclusion

In this lab, we successfully explored and implemented three fundamental looping constructs in C++:

• The while loop checks the condition before executing the loop body. It is useful when the number of iterations is unknown and depends on a condition.

• The do...while loop executes the loop body at least once before checking the condition. It is ideal for scenarios where at least one execution is required, such as menu-driven programs or password validation.

• The nested while loop allows complex pattern generation and multi-level iteration by placing one loop inside another.

• All three loop types are essential tools for controlling program flow and solving repetitive tasks efficiently.

Understanding the differences between these loops and knowing when to apply each one is crucial for writing efficient, readable, and maintainable C++ programs.